Rewriting Binaries for Bug Hunting

Reproducing RetroWrite for Binary-Only Fuzzing and Sanitization

SIL7165: Networks & System Security

Semester 2, 2025-26

Chirag Suthar (2025MCS2105)
Haleel Sada (2025MCS2741)

March 2026

Introduction

Why do we need this?

The Problem

- Most software is closed-source — we only get the compiled binary
- Binaries have hidden memory bugs (buffer overflow, use-after-free, etc.)
- Hackers exploit these bugs to steal data or take control of systems

Existing Solutions Are Too Slow

- ASan and AFL are great bug-finding tools, but need source code
- For binaries: QEMU (10–100x slower), Valgrind (2–300x slower)
- Slow = fewer test inputs = fewer bugs found

RetroWrite's Idea

- Convert binary back to editable assembly using relocation info
- Add security checks (ASan/AFL) to the assembly
- Reassemble into a new binary — runs at near-native speed!

What is RetroWrite?

IEEE S&P 2020 | 300+ citations

RetroWrite = Static Binary Rewriter

- Takes a compiled binary as input (no source code needed)
- Produces reassembleable assembly using relocation info
- Allows adding security checks like ASan and AFL coverage
- Reassembles into a new working binary

Key Insight

- Modern 64-bit PIC binaries have relocation info
- RetroWrite uses these to identify addresses exactly
- No guessing / no heuristics = no broken binaries!

The Pipeline

Binary → Disassemble → Symbolize
Add Checks → Reassemble → Safe Binary

Paper Claims

- AFL: 4.2x–5.6x faster than QEMU-based fuzzing
- ASan: 3x faster than Valgrind, finds 80% more bugs

Background

Key technical concepts

Binary Disassembly

- Converting machine code back to assembly
- Tools: IDA Pro, Ghidra, Capstone (used by RetroWrite)

Position-Independent Code (PIC)

- Binaries that can load at any address (ASLR)
- Contain relocation entries — tell where addresses are
- All modern Linux binaries are PIC by default

Address Sanitizer (ASan)

- Memory error detector by Google
- Shadow memory: 1 byte tracks 8 bytes of real memory
- Checks every access; reports freed/OOB memory

Coverage-Guided Fuzzing (AFL)

- Feed random inputs, track code coverage
- Prefer inputs that reach new code paths
- Needs fast execution to test millions of inputs

Background

How binary rewriting works

Two Approaches to Binary Instrumentation

1. Dynamic Binary Translation (DBT)

- Translate code at runtime, block by block
- Examples: QEMU, Valgrind, Intel PIN, DynamoRIO
- Works on any binary, but very slow (10–100x overhead)

2. Static Binary Rewriting

- Modify binary offline, before execution
- Much faster (no runtime translation overhead)
- Challenge: correctly identify addresses vs numbers

The Symbolization Problem

- When rewriting, you need to know which values are addresses
- If you move code/data, addresses must be updated
- Getting this wrong = broken binary

✘ Old tools (Uroboros, ramblr) guess with heuristics — often wrong

✔ **RetroWrite uses relocations — always correct for PIC binaries!**

Related Work

Comparison of existing approaches

Tool	Type	Sound?	Fast?	Scalable?	Key Limitation
QEMU	DBT	Yes	No (10–100x)	Yes	Very slow for fuzzing
Valgrind	DBT	Yes	No (2–300x)	Yes	Too slow for fuzzing
Intel PIN	DBT	Yes	No (10–20%)	Yes	Runtime overhead
DynamoRIO	DBT	Yes	No (10–20%)	Yes	Runtime overhead
Uroboros	Static	No	Yes	No	Wrong heuristics
ramblr	Static	No	Yes	Partial	Still uses heuristics
DynInst	Static	Partial	Yes	Partial	Unsound
RetroWrite	Static	YES	YES	YES	Only 64-bit PIC

RetroWrite is the first tool to achieve soundness + speed + scalability together.

Problem Statement

System Model & Threat Model

System Model

- Target: 64-bit Linux PIE binary (closed-source)
- RetroWrite: Disassembles, symbolizes, produces assembly
- Instrumentation Pass: Adds ASan / AFL coverage
- Assembler: Recompiles to working binary
- AFL Fuzzer: Tests with millions of random inputs

Threat Model

- Attacker: Can send crafted inputs to the program
- Goal: Exploit memory bugs for code execution
- Defender: Security analyst testing binary software
- Goal: Find bugs before attackers, using RetroWrite

Key Challenges

- Correctly symbolizing all addresses (no room for error)
- Not breaking ABI (red zone, calling conventions)
- Finding free registers for instrumentation code

Proposed Solution

The RetroWrite pipeline

1 Preprocessing

- Load binary sections (.text, .data), read relocations
- Disassemble using Capstone, build CFG

2 Symbolization

- Replace hardcoded addresses with labels
- Uses relocations for sound symbolization
- Three types: control-flow, PC-relative, data

3 Instrumentation

- ASan: Shadow memory check before every read/write
- AFL: Coverage counter at every basic block

4 Optimization

- Register liveness analysis to minimize overhead
- Only save registers actually in use

5 Reassembly

- Compile modified assembly with GCC/Clang
- Output: instrumented binary ready for testing

Proposed Solution

How ASan instrumentation works

```
                Shadow Memory Check (before every memory access):
Original:
  mov [rax+15], 0x58        ; write to memory

With ASan:
  lea  rdi, [rax+15]        ; get target address
  shr  rdi, 3               ; lookup shadow memory
  cmp  [rdi+offset], 0      ; is it poisoned?
  jnz  __asan_report        ; YES = BUG FOUND!
  mov  [rax+15], 0x58       ; NO = safe, proceed
```

Shadow Memory

1 byte tracks 8 bytes of real memory

Only 12.5% memory overhead

Redzone Policy Comparison

Memory Region	Source ASan	ASan-retrowrite
Heap	Individual object	Individual object (same)
Stack	Individual var, 32-byte	Stack-frame, 8-byte
Global	Individual, padded 64-byte	No redzone

Proposed Solution

How AFL coverage instrumentation works

```
At every basic block, RetroWrite inserts:
inc byte [coverage_map + BLOCK_ID]
; "I visited this block!"
```

How AFL Uses This

- Run program with input, read coverage map
- If input reaches NEW code paths → keep & mutate
- If no new paths → discard
- Repeat millions of times to explore deep paths

Why RetroWrite is Better — Speed Comparison

Method	Speed (exec/sec)	% of Source
Source AFL (best possible)	4,790	100%
RetroWrite AFL (binary-only)	4,244	88.6%
QEMU AFL (old way)	~800	~17%

RetroWrite achieves 88.6% of source-level speed — assembly looks like compiler output, compatible with afl-gcc directly.

Completed Evaluation

What we did

Experimental Setup

Machine: Ubuntu Linux, x86-64 | Tools: RetroWrite (GitHub), AFL++, GCC, Clang | Libraries: Capstone, pyelftools

Experiment 1
ASan Bug Detection

- Used heap.c demo with heap overflow + use-after-free bugs
- Compiled as PIE binary
- Instrumented with RetroWrite ASan

Experiment 2
Real-World Binary Rewriting

- Rewrote bzip2 1.0.8 (106KB real compression tool)
- Verified output is identical to original

Experiment 3
AFL Fuzzing Performance

- Custom fuzz target with intentional bugs
- Compared RetroWrite AFL vs source-level AFL speed

Completed Evaluation

Results

ASan Bug Detection Results

Bug Type	Original Binary	RetroWrite + ASan
Heap buffer overflow	Not detected	**DETECTED**
Use-after-free	Not detected	**DETECTED**

AFL Fuzzing Speed Comparison

Method	Speed (exec/sec)	% of Source
Source AFL (best possible)	4,790	100%
RetroWrite AFL (binary-only)	4,244	88.6%
QEMU AFL (old way)	~800	~17%

bzip2 Rewrite Result

Successfully rewritten (106KB)

Output identical to original

Key Takeaways

- ✔ ASan catches real bugs without source code
- ✔ Fuzzing speed within 12% of source-level
- ✔ Much faster than QEMU/Valgrind alternatives

Completed Evaluation

Comparison with existing tools

Tool	Needs Source?	Runtime Overhead	Bug Detection
Source ASan	Yes	1.65x	Best
RetroWrite ASan	No	~1.65x	Good (80% of source)
Valgrind memcheck	No	20–300x	Lower
QEMU + AFL	No	10–100x	Low throughput
RetroWrite + AFL	No	~1x	Near source-level

Our Results Confirm Paper Claims

✔ ASan catches real bugs in binaries without source code
✔ Binary rewriting produces correct output on real-world software
✔ Fuzzing speed is within 12% of source-level AFL
✔ Much faster than QEMU/Valgrind alternatives

Planned Evaluation

What we plan to do for endterm

Extension 1

Fix rep Prefix ASan

- rep movsb/stosb (memcpy/memset) not instrumented currently
- Marked as TODO in source code
- Plan: Add shadow memory checks for rep region boundaries

Extension 2

Basic Block Coverage Pass (x64)

- ARM64 version exists, x64 version is missing
- Plan: Insert coverage counter at each basic block

Extension 3

Stack Canary Insertion

- Add canaries to binaries compiled without -fstack-protector
- Push canary at function entry, verify before every ret
- Catches stack buffer overflows

Evaluation Plan

- Test rep fix on programs with memcpy/memset overflows
- Compare coverage pass accuracy with source-level coverage
- Test canary insertion on known stack overflow programs
- Measure performance overhead of each extension

Summary

What We Studied

- RetroWrite: static binary rewriting framework (IEEE S&P 2020)
- Uses relocation info in PIC binaries for sound symbolization
- Adds ASan (bug detection) and AFL (fuzzing) to binary-only software

What We Did

- Set up RetroWrite and reproduced core functionality
- Detected real memory bugs (heap overflow, use-after-free)
- Rewrote bzip2 with correct output
- Achieved 88.6% of source-level AFL fuzzing speed

What We Plan

- Fix rep prefix ASan limitation
- Add coverage pass for x64
- Implement stack canary insertion

Key Takeaway

For 64-bit PIC binaries, static rewriting with relocations gives source-level security tools without source code, at near-native speed.

References

[1] S. Dinesh et al., "RetroWrite: Statically Instrumenting COTS Binaries for Fuzzing and Sanitization," IEEE S&P, 2020.

[2] K. Serebryany et al., "AddressSanitizer: A Fast Address Sanity Checker," USENIX ATC, 2012.

[3] M. Zalewski, "American Fuzzy Lop (AFL)," 2017.

[4] F. Bellard, "QEMU, a Fast and Portable Dynamic Translator," USENIX ATC, 2005.

[5] N. Nethercote and J. Seward, "Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation," PLDI, 2007.

[6] C.-K. Luk et al., "Pin: Building Customized Program Analysis Tools with Dynamic Instrumentation," PLDI, 2005.

[7] D. Bruening et al., "An Infrastructure for Adaptive Dynamic Optimization," CGO, 2003.

[8] S. Wang et al., "Reassembleable Disassembling (Uroboros)," USENIX Security, 2015.

[9] R. Wang et al., "Ramblr: Making Reassembly Great Again," NDSS, 2017.

[10] A. Bernat and B. Miller, "Anywhere, Any-Time Binary Instrumentation (Dyninst)," PASTE, 2011.